

LLM Tools

This work is distributed under the Creative Commons Attribution 4.0 International License. The copyright for the components of this work owned by parties other than the author (Luca Cotti) and the University of Brescia must be respected.

While preparing this work, the author used Claude Sonnet 4.6 to improve readability and language. After using it, the author reviewed and modified the content as necessary.

1	Setup	2
1.1	Prerequisites	2
1.2	Environment configuration	2
1.3	Examples	3
2	Background	3
2.1	Retrieval-Augmented Generation	3
2.2	LLM Tools	3
2.3	RDF Stores	3
3	Task 1: SPARQL-based RAG	4
3.1	Goal	4
3.2	Functions to implement	4
3.3	Pipeline entry point	4
3.4	What to inspect	5
4	Task 2: Hybrid Retrieval	5
4.1	Goal	5
4.2	Preparation	5
4.3	Strategy A: Structured SPARQL	5
4.4	Strategy B: Text/keyword retrieval	5
4.5	(Optional) Strategy C: Embedding-based retrieval	6
4.6	Comparison	6
5	Task 4: RAG Agent with Tool Calling	6
5.1	Goal	6
5.2	Functions to implement	7
5.3	Agent loop	7
5.4	What to observe	7

In this laboratory you will build, step by step, a question-answering system that combines a LLM with information extracted from a knowledge graph.

By the end of the session you will have implemented:

- two retrieval strategies (structured SPARQL, keyword search) applied to a knowledge graph;

- an agent that uses tool calling to select the appropriate retrieval strategy.

For these tasks you will use a small version of Qwen 3.5. Due to its limited parameter count, the model will often produce inaccurate or incomplete answers when questioned on specific facts. The objective of this lab is to improve its accuracy by supplying it with relevant information retrieved at query time, rather than relying on what the model has memorised during training. The model also supports a reasoning (“thinking”) mode; it is recommended to keep it disabled during this lab as it significantly increases response latency.

You will interact with the LLM through the LiteLLM library (<https://docs.litellm.ai/docs/#litellm-python-sdk>), which provides a minimal but consistent interface across a variety of models.

1 Setup

1.1 Prerequisites

Clone the starter project from the GitHub repository (https://github.com/LucaCtt/lab_llm_tools). Please take a moment to familiarize yourself with the structure and contents of the repository.

`uv` is used to manage the Python environment. If you have not already installed it, follow the instructions at <https://docs.astral.sh/uv/getting-started/installation/>.

1.2 Environment configuration

To install the required dependencies, run the following command in the project root:

```
uv sync
```

This creates a `.venv` directory and installs all dependencies declared in `pyproject.toml`.

Note for Visual Studio Code users. Make sure to [select the Python environment](#) in `.venv` after running the command above.

Copy `.env.default` to `.env` and fill in your LiteLLM key:

```
cp .env.default .env
# then edit .env and set:
# LITELLM_API_KEY=<your key>
```

The model endpoint and all generation parameters are declared in `src/lab/settings.py` via [pydantic-settings](#). Settings can be overridden with environment variables without changing the code.

1.3 Examples

Before starting the tasks, take a look at the provided examples:

File	What it shows
<code>example_1_litellm.py</code>	Basic LLM call with streaming
<code>example_2_function_calls.py</code>	Tool calling with mock functions
<code>example_3_wikidata.py</code>	Wikidata querying and saving to file

To run the examples you can use `uv run`:

```
uv run python *path of the script*
```

2 Background

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is a pattern in which an LLM is provided with external knowledge retrieved at query time, rather than relying solely on the information learned during training.

In this lab, retrieval is instead performed through *tools*. Rather than injecting context automatically before each call, the model is given a set of retrieval functions and decides autonomously which one to invoke, with which arguments, and whether to call multiple tools in sequence. This design is closer to an *agentic* RAG pattern, in which the model actively directs its own information gathering.

2.2 LLM Tools

Tool calling (also called [function calling](#)) allows a language model to request the execution of external functions during a conversation. Instead of generating a plain text response, the model outputs a structured request specifying a function name and its arguments. The host application executes the function and returns the result to the model, which then produces a final answer. You can see an example of this in the `example_2_function_calls.py` file.

2.3 RDF Stores

An RDF store is a database designed to persist and query RDF triples. Unlike relational databases, which organise data into tables, RDF stores are optimised for graph-shaped data: arbitrary entities connected by typed relationships.

In a real-world context, you'd use a production system such as [Neo4j](#) and [GraphDB](#). In this lab, for simplicity, the RDF store will be a plain `.ttl` file (Turtle format) loaded into an in-memory `rdflib.Graph`. You will populate the graph at runtime by fetching

triples from Wikidata and serialise it to disk so that it survives between runs.

3 Task 1: SPARQL-based RAG

File: `src/lab/task_1_sparql.py`

3.1 Goal

Build the RAG pipeline described in Section 3.2. You are not required to build a general text-to-SPARQL system. Work on a single narrow topic of your choice and make your pipeline answer at least three questions from that topic reliably.

3.2 Functions to implement

Function	What it should do
<code>generate_sparql(question)</code>	Call the LLM with with an appropriate prompt and return the SPARQL query string.
<code>query_wikidata(sparql)</code>	Send a GET request to the Wikidata SPARQL endpoint. Return a list of <code>{variable: value}</code> dicts. Handle HTTP errors and empty results gracefully.
<code>verbalize(results)</code>	Convert result rows into short natural-language sentences. Return "No relevant information was found." for empty input.
<code>answer(question, context)</code>	Call the LLM a second time with the verbalized context. Instruct it to answer using only the retrieved facts.

3.3 Pipeline entry point

The `ask(question)` function is already provided. It calls each stage in order and prints all intermediate outputs. You should see something like:

```
=====
QUESTION: Who designed the Python programming
         language?
=====
[SPARQL]
SELECT ?designerLabel WHERE {
  wd:Q37100 wdt:P178 ?designer .
  SERVICE wikibase:label { ... }
}

[RAW RESULTS] 1 row(s)

[CONTEXT]
Python was developed by Guido van Rossum.

[ANSWER]
Python was designed by Guido van Rossum.
```

3.4 What to inspect

A wrong final answer may be caused by:

- an incorrect SPARQL query (wrong entity ID or property);
- empty or irrelevant Wikidata results;
- poor verbalisation of the retrieved facts;
- weak prompting in the answer step.

We will see alternatives that address the first two cases in the next tasks. For now, try using appropriate prompt techniques to fix any issue.

4 Task 2: Hybrid Retrieval

File: `src/lab/task_2_hybrid_retrieval.py`

4.1 Goal

Implement different retrieval strategies over a local RDF graph, then compare their outputs side by side for the same question.

4.2 Preparation

Please run the `example_3_wikidata.py` script, either with the original example SPARQL query or a custom one of your choice. You can also use queries from previous lab sessions if you prefer.

4.3 Strategy A: Structured SPARQL

Function: `retrieve_sparql(graph, question)`

Use the LLM to generate a SPARQL query, but execute it against the *local* graph instead of Wikidata. This exposes a key limitation: the LLM will generate Wikidata URIs (`wd:`, `wdt:`) that do not exist in the local graph. You will need to either adapt the query to use the `LAB:` namespace, or write a fixed local query based on the question's keywords.

4.4 Strategy B: Text/keyword retrieval

Function: `retrieve_text(graph, question)`

Extract one or two keywords from the question and search for them in literal values using the `FILTER(Regex(...))`. For more info: <https://en.wikibooks.org/wiki/SPARQL/FILTER>

This approach is tolerant of URI mismatches but may return noisy results.

4.5 (Optional) Strategy C: Embedding-based retrieval

An embedding is a function that maps a discrete object such as a word, a sentence, a graph node, or an entire document to a dense vector of real numbers. The key property is that objects with similar meaning or structure are mapped to vectors that are close in the embedding space, typically measured by cosine similarity. Text embeddings are central to vector-based RAG: a query and a set of candidate documents are all embedded, and retrieval reduces to a nearest-neighbour search.

A knowledge graph does not have a single canonical representation as a vector. Several strategies exist, each making different trade-offs. Some basic techniques include:

- **Subgraph text.** Serialise of a relevant subgraph to a text format such as Turtle or a list of natural-language sentences, then embed the resulting string.
- **Per-triple embedding.** Verbalize each triple individually and embed each sentence. Retrieval then selects the most relevant triples rather than the most relevant document.
- **Per-entity embedding.** Collect all triples for a given entity, verbalize them as a short paragraph, and embed the paragraph.

You can choose any of these techniques. You can get the embedding of a string through the `_get_embedding(text)` function. Once you have computed the embedding of a subgraph, you can compare it to the embedding of the question through the provided `_cosine_similarity(a,b)` function.

4.6 Comparison

The `compare(graph, question)` function is already provided. It runs all three strategies on the same question and prints each context before calling `answer_with_context()`.

Run it for at least two questions and note:

- Which strategy returns the most relevant context?
- Which strategy fails silently (empty results)?
- Does the final answer differ across strategies?

5 Task 4: RAG Agent with Tool Calling

File: `src/lab/task_4_rag_agent.py`

5.1 Goal

Wrap the three retrieval strategies as LLM tools and let the model decide which one to call. This follows exactly the same tool-calling pattern as `example_2_function_calls.py`, but the tools now perform real retrieval over the local RDF knowledge base.

5.2 Functions to implement

Function	What it should do
<code>run_agent(question, graph)</code>	Call the LLM with the retrieval tools defined in the previous task and a knowledge graph.

5.3 Agent loop

Follow the same structure as `example_2_function_calls.py`:

1. Build `messages` with the system prompt and user question.
2. First call: `litellm.completion(..., tools=tools, stream=False)`.
3. Extract `tool_calls` from the response.
4. For each tool call: execute the function, append both the assistant message and the tool result to `messages`.
5. Second call: `litellm.completion(..., stream=True)` for the final streamed answer.

5.4 What to observe

For each test question, note which tool the model chose and why. Does the model always pick the most appropriate strategy? Try rephrasing the same question to see if the tool selection changes.